Q(V, ϕ) parametrization — method

ETL LGAD charge vs bias and fluence

October 5, 2026

This note describes, step by step, the laws used to parametrize the charge Q collected by ETL LGADs as a function of the bias V and the neutron–equivalent fluence ϕ. It documents the code in this repository (`data_io.py`, `model.py`, `plots.py`, `config.py`).

1 Data and pre-processing

Measurements live in `parametrization_data.xlsx`, one *sheet per production* (e.g. `HPK`; `FBK` later). Inside a sheet each sensor scan is a block of three columns V, Q, Res; the block label carries the sensor name and the fluence (e.g. `HPK6 S4-B - 1e15`). The reader extracts:

- **production** = sheet name;
- **fluence** parsed from the label (0 if none is written);
- **setup** = `TB` if the label mentions a test beam, else `Beta`.

Pooling. For a given production and fluence, the (V, Q) points of all `Beta` sensors are *pooled* into a single data set and fitted together; test–beam (`TB`) points are shown only as a cross–check and are never fitted. Within one production sensors of nominally equivalent generations may be pooled across fluence (e.g. HPK6 and HPK2).

Uncertainties. The raw data has no error bars, so constant values are assigned (editable in `config.py`):

$$\sigma_Q = 1\,\text{fC (switchable to 2 fC)}, \qquad \sigma_V = 2\,\text{V}, \qquad \sigma_\phi = 0.10\,\phi. \tag{1}$$

A constant σ_Q does not bias the central fit values; it only sets the weighting and the reported parameter errors.

2 Step 1 — charge vs bias at fixed fluence

At each fluence the pooled $Q(V)$ points are fitted with

$$\boxed{Q(V) = p_0 + p_1\,V + \exp\big(p_2 + p_3\,V\big)} \tag{2}$$

a linear term plus an exponential (gain–like) rise.

Analytic seeding (no manual tuning). ROOT's default start values make this fit unstable, which previously required hand–tuned parameter limits. Instead the seed is computed from the data (`auto_seed_QV`):

1. sort by V; fit a straight line to the lowest ∼40% of the points to get the baseline (p_0, p_1);
2. form the residual $R(V) = Q - (p_0 + p_1 V)$ and, on the points with $R > 0$, fit $\ln R = p_2 + p_3 V$ to get (p_2, p_3).

With this seed the fit converges on its own.

Physical bounds. The only constraints are *uniform* (the same for every fit, not per point):

$$p_1 \geq 0, \qquad p_3 \geq 0, \tag{3}$$

i.e. the collected charge does not decrease with bias. The fit is performed in ROOT (`TGraphErrors::Fit` of Eq. 2); if ROOT fails to return a valid result, a `scipy.optimize.curve_fit` with the same seed is used as a silent fallback.

The outputs are the four parameters $p_k(\phi_j)$ and their errors at each measured fluence ϕ_j, with the fit χ^2/ndf.

3 Step 2 — parameters vs fluence

Each parameter p_k is then modelled as a function of ϕ. Four laws are available (`config.PARAM_MODE`); let $\{(\phi_j, p_{k,j}, \sigma_{k,j})\}$ be the points from Step 1.

(i) Node–exact interpolation `interp` (default). Piecewise–linear through the measured points, with linear extrapolation beyond the end nodes:

$$p_k(\phi) = p_{k,j} + \frac{p_{k,j+1} - p_{k,j}}{\phi_{j+1} - \phi_j}\,(\phi - \phi_j), \qquad \phi \in [\phi_j, \phi_{j+1}]. \tag{4}$$

By construction it reproduces every measured fluence exactly, so the predicted $Q(V, \phi)$ matches the Step–1 fit at each ϕ_j.

(ii) Linear `lin`. Weighted least squares,

$$p_k(\phi) = m_k\,\phi + q_k, \qquad \text{weights } w_j = 1/\sigma_{k,j}. \tag{5}$$

(iii) Exponential `exp`.

$$p_k(\phi) = a_k + \exp\big(b_k\,\phi + c_k\big), \tag{6}$$

fitted in ROOT with an analytic seed (a_k near the asymptote, (b_k, c_k) from a log fit of $p_k - a_k$).

(iv) Automatic `auto`. If there are fewer than `MIN_PTS_FOR_EXP` $= 4$ fluences, use `lin`. Otherwise fit both `lin` and `exp` and keep the one with the smaller Akaike information criterion,

$$\mathrm{AIC} = \chi^2 + 2k, \tag{7}$$

with k the number of free parameters (2 for `lin`, 3 for `exp`). This is decided independently for each p_k.

Why `interp` is the default here. The form of Eq. 2 is partly degenerate: p_0, p_1, p_2 are strongly anti–correlated, so while the *curve* $Q(V)$ is well constrained the *individual* parameters are not and scatter from fluence to fluence. As a result the measured $p_k(\phi)$ do not follow a smooth low–order law, and a global `lin`/`exp` fit of p_2, p_3 — to which Q is exponentially sensitive — can mispredict the charge badly. The node–exact `interp` avoids this by reproducing the data at every measured fluence; `auto`/`exp` remain available when a smooth, extrapolable surface is preferred.

4 Combined surface and its inverse

Substituting the parameter laws into Eq. 2 gives the two–dimensional surface

$$\boxed{Q(V, \phi) = P_0(\phi) + P_1(\phi)\,V + \exp\big(P_2(\phi) + P_3(\phi)\,V\big)} \tag{8}$$

with $P_k(\phi)$ one of the laws above. When every P_k is `lin` or `exp` this is exported as a single closed–form ROOT `TF2` (`build_QVphi_TF2`); for `interp` the surface is evaluated numerically (`Q_of_V_phi`).

Bias for a target charge. To get the bias needed to reach a charge $Q^\star$ at a fluence ϕ, the equation $Q(V,\phi) = Q^\star$ is inverted numerically (`V_for_Q`): $Q(V,\phi)$ is scanned on a dense V grid and the *highest* upward crossing of $Q^\star$ is returned (robust to any non–monotonic behaviour of a degenerate fit); `NaN` if $Q^\star$ is never reached.

5 The analytic model: reduced two–step parametrization

The four–parameter form of Eq. 2 is partly degenerate (p_0, p_1, p_2 anti–correlated), so the individually fitted $p_k(\phi)$ scatter and no smooth law describes them well. The remedy is to remove the degenerate parameters physically:

- fix $p_0 = Q_{\text{floor}} = 0.55\,\text{fC}$ (`config.Q_FLOOR`), the charge a diode collects once the gain layer is gone (the no–gain, bulk MIP charge);
- drop the linear term p_1 (it is redundant: removing it leaves the per–fluence χ^2 essentially unchanged).

Step A (per fluence). Fit the *reduced* curve

$$Q(V) = Q_{\text{floor}} + \exp\big(p_2 + p_3\,V\big), \tag{9}$$

with only two free parameters. With p_0, p_1 gone, $p_2(\phi)$ and $p_3(\phi)$ come out smooth and monotonic.

Step B (vs fluence). Give each an *analytic* law $p_2(\phi)$, $p_3(\phi)$ (linear or exponential, chosen by AIC as in Sec. 3, laws (ii)–(iv); `interp` is never used here). The model is then the single analytic surface

$$\boxed{Q(V,\phi) = Q_{\text{floor}} + \exp\big(p_2(\phi) + p_3(\phi)\,V\big)} \tag{10}$$

one continuous law with no node kinks that, as the gain term vanishes at high fluence, tends to Q_{floor} rather than to zero. The bias for a target charge $Q^\star > Q_{\text{floor}}$ is explicit:

$$V(\phi) = \frac{\ln\big(Q^\star - Q_{\text{floor}}\big) - p_2(\phi)}{p_3(\phi)}. \tag{11}$$

With linear laws $p_2(\phi) = a_0 + a_1\phi$, $p_3(\phi) = b_0 + b_1\phi$ this is exactly $Q = Q_{\text{floor}} + \exp[(a_0 + a_1\phi) + (b_0 + b_1\phi)V]$. This model (`model.GlobalSurface`, `model.fit_global`) is what the prediction and operating–point plots use; the full four–parameter per–fluence fits of Sec. 2 remain available as diagnostics.

6 Operating point with the SEB limit

Each production has a single–event–burnout ceiling V_{SEB} (`config.V_SEB`; HPK 550 V, FBK 600 V) above which it cannot be operated. For a target charge $Q^\star$ (default `PLATEAU_CHARGE` = 15 fC):

$$\text{bias needed:} \quad V(\phi) = \min\big(V_{\text{SEB}},\, V_{\text{for}\,Q^\star}(\phi)\big), \tag{12}$$

$$\text{charge obtainable:} \quad Q_{\text{obt}}(\phi) = \min\big(Q^\star,\, Q(V_{\text{SEB}},\phi)\big). \tag{13}$$

Below the "SEB knee" the target is reached within V_{SEB} so $Q_{\text{obt}} = Q^\star$ (flat); beyond it the device is stuck at V_{SEB} and delivers the smaller charge $Q(V_{\text{SEB}},\phi)$, which falls with fluence towards Q_{floor}. Here $Q(V_{\text{SEB}},\phi)$ is taken from the global surface of Sec. 5, so the curve is a single smooth trend and the charge converges to $Q_{\text{floor}} = 0.55\,\text{fC}$, not to zero. These are the solid (charge, left axis) and dashed (bias, right axis) curves of the operating–point figure.

7 Caveats

- Parameter degeneracy (above) makes the individual p_k noisy; prefer the *curves*/predictions over the individual parameter values.
- Extrapolation in V beyond the measured range, and in ϕ beyond the last measured fluence, is model dependent and should be treated with care.
- With `interp` the surface is only as smooth as the data; for a smooth analytic law use `auto`/`exp`, accepting that it will not pass exactly through every measured fluence.